

LAPORAN UAS MINI PROJECT

Communication Protocol

Credit Score Mock API

Enterprise Credit Scoring REST API dengan Observability Dashboard



Disusun Oleh :

M. Maulana Khaerul Anam
25120500032

Dosen Pengampu :

Haikal Shiddiq S.Kom, M.T.
Communication Protocol

Program Studi Sains Data Fakultas Ilmu Komputer

Universitas Cakrawala 2026

Problem Statement

Latar Belakang

Dalam proses pengajuan kredit, lembaga keuangan membutuhkan sistem yang mampu melakukan penilaian risiko terhadap calon peminjam secara cepat, konsisten, dan terdokumentasi. Sistem tersebut harus mampu menerima data pengajuan, melakukan proses penilaian, menghasilkan keputusan kredit, serta menyediakan mekanisme monitoring terhadap seluruh aktivitas API.

Pada mini project ini dikembangkan sebuah aplikasi CreditScore Mock API yang mensimulasikan proses credit scoring menggunakan REST API. Sistem dirancang agar dapat diuji menggunakan browser maupun Postman serta menyediakan fitur observability berupa audit log, Request ID, Correlation ID, response time, dan status code.

Tujuan

- Membangun REST API sederhana menggunakan TanStack Start.
- Mengimplementasikan komunikasi HTTP berbasis JSON.
- Melakukan pengujian endpoint menggunakan Postman.
- Menyediakan observability terhadap seluruh request API.
- Menganalisis keberhasilan maupun kegagalan komunikasi protokol.

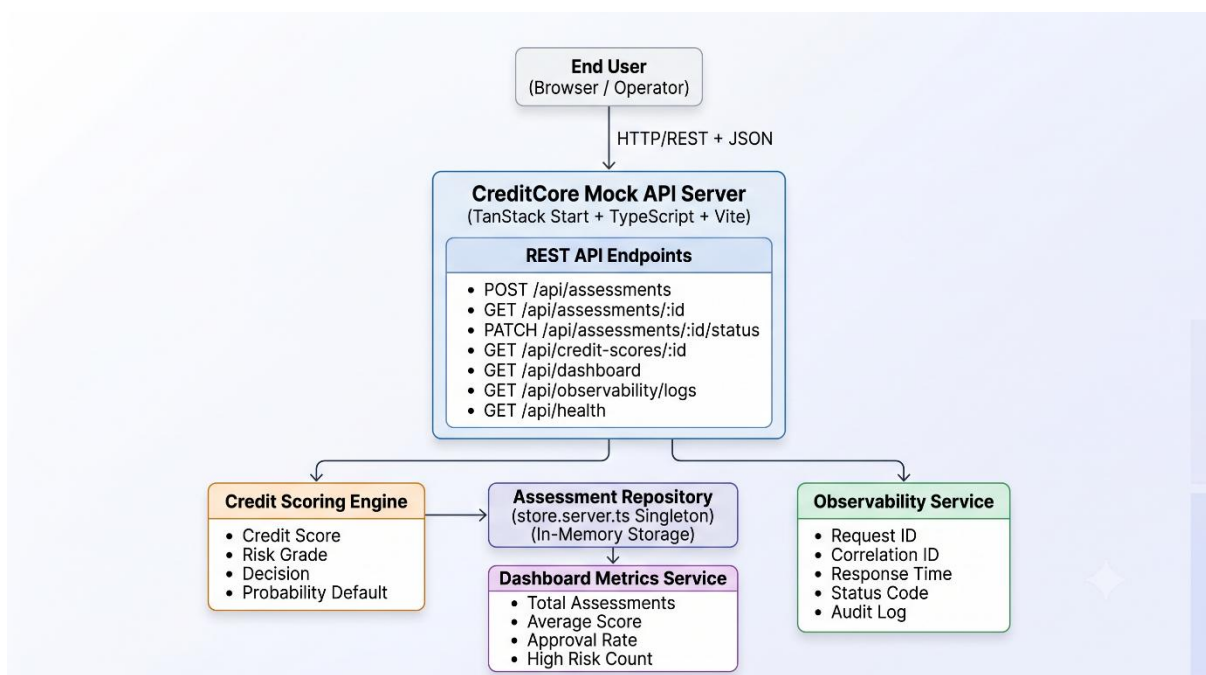
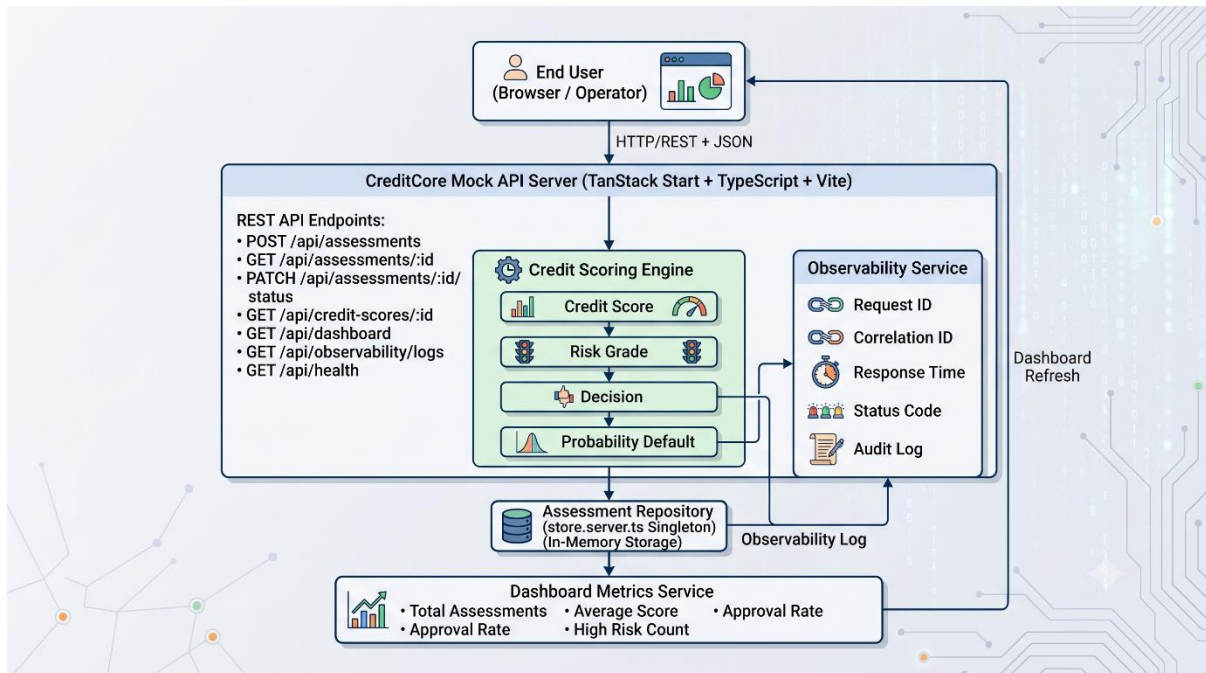
Use Case yang Dipilih

Model Scoring Mock API

Alasan pemilihan:

- Merepresentasikan implementasi REST API yang umum digunakan pada industri fintech.
- Memiliki alur request-response yang jelas.
- Mudah diuji menggunakan Postman.
- Mendukung observability.
- Memenuhi seluruh requirement UAS.

Architecture Canvas dan Data Flow Diagram



Penjelasan Architecture

Sistem terdiri dari beberapa komponen utama yaitu:

- End User sebagai client

- REST API Server
- Credit Scoring Engine
- Assessment Repository
- Dashboard Metrics Service
- Observability Service

Client mengirim request menggunakan HTTP/REST. Server memproses request melalui Credit Scoring Engine, kemudian menyimpan hasil assessment ke repository. Dashboard membaca data dari repository untuk menampilkan statistik sedangkan Observability Service mencatat seluruh aktivitas API.

Protocol Selection

Protokol yang Digunakan — REST API

REST digunakan sebagai media komunikasi utama antara client dan server.

Alasan:

- Stateless
- Mudah diimplementasikan
- Didukung Postman
- Menggunakan HTTP standar
- Format JSON mudah dipahami

HTTP

Method yang digunakan:

- GET
- POST
- PATCH

JSON

Seluruh request dan response menggunakan format JSON. Contoh:

```
{
  "accountId": "ACC-2026-0001",
  "borrowerName": "Anam",
  "requestedAmount": 10000000
}
```

Mengapa Tidak Menggunakan WebSocket?

WebSocket digunakan untuk komunikasi real-time dua arah. Project ini hanya membutuhkan komunikasi request-response sehingga REST API lebih sesuai.

Mengapa Tidak Menggunakan MQTT?

MQTT lebih cocok untuk IoT dan sensor. Project credit scoring tidak membutuhkan komunikasi publish-subscribe.

Mengapa Tidak Menggunakan gRPC?

gRPC memiliki performa tinggi namun membutuhkan konfigurasi lebih kompleks. Untuk mini project ini REST API sudah memenuhi kebutuhan.

Mengapa Tidak Menggunakan n8n?

n8n bersifat opsional sesuai ketentuan UAS. Karena seluruh kebutuhan komunikasi sudah dapat dipenuhi menggunakan REST API dan Postman, maka workflow automation tidak digunakan. Jika ada pengembangan mungkin akan dilakukan otomatisasi dengan n8n.

Endpoint Design

1. Business API

Method	Endpoint	Fungsi
POST	/api/assessments	Membuat assessment baru
GET	/api/assessments/:id	Detail satu assessment berdasarkan id
PATCH	/api/assessments/:id/status	Update status
GET	/api/credit-scores/:id	Melihat credit score
GET	/api/assessments	Melihat seluruh daftar assessments

2. Infrastucture API

Method	Endpoint	Fungsi
GET	/api/health	Memeriksa status server
GET	/api/dashboard	Menampilkan statistik dashboard
GET	/api/observability/logs	Menampilkan audit log
GET	/api/simulation	Melihat status simulasi
PUT	/api/simulation	Mengaktifkan simulasi error
PUT	/api/simulation/reset	Mengembalikan simulasi ke kondisi normal

Pada implementasi mini project ini terdapat perbedaan antara endpoint yang tersedia pada **Postman Collection** dan **CreditCore Demo App**.

Demo App hanya menampilkan endpoint yang berkaitan langsung dengan proses bisnis (business flow), seperti:

- POST /api/assessments
- GET /api/assessments/:id
- PATCH /api/assessments/:id/status
- GET /api/credit-scores/:id
- GET /api/dashboard
- GET /api/observability/logs

- GET /api/health

Sementara itu, **Postman Collection** juga menyediakan endpoint tambahan:

- GET /api/simulation
- PUT /api/simulation
- PUT /api/simulation/reset

Endpoint tersebut **tidak ditampilkan pada antarmuka aplikasi** karena fungsinya bukan bagian dari proses bisnis, melainkan sebagai **alat pengujian (testing utility)**.

Endpoint simulation digunakan untuk:

- Mensimulasikan kondisi error seperti HTTP 500 Internal Server Error.
- Mengembalikan kondisi server ke keadaan normal setelah simulasi selesai.
- Mempermudah pengujian skenario failure tanpa harus mengubah kode aplikasi.
- Mendukung pengujian reliability dan error handling sesuai kebutuhan UAS.

Dengan demikian, Postman Collection memiliki cakupan endpoint yang lebih lengkap dibandingkan Demo App karena ditujukan sebagai alat pengujian API, sedangkan Demo App difokuskan pada fitur yang digunakan oleh pengguna (business workflow).

Request Header

```
Content-Type: application/json
```

Request Body

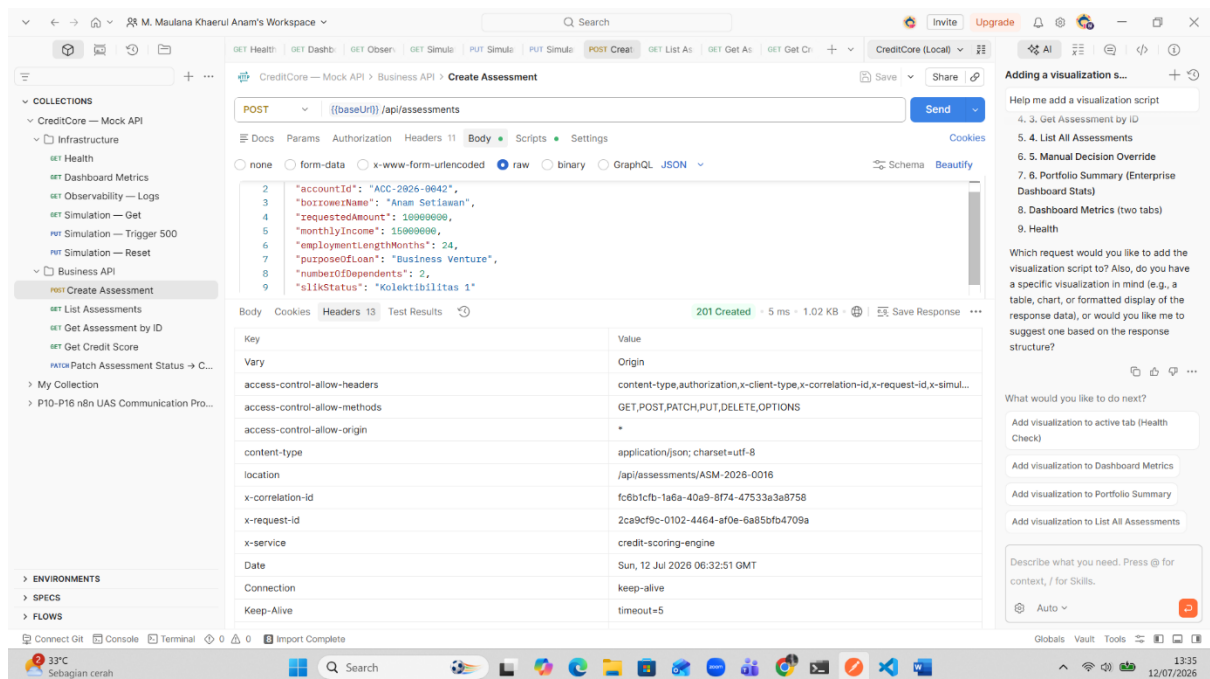
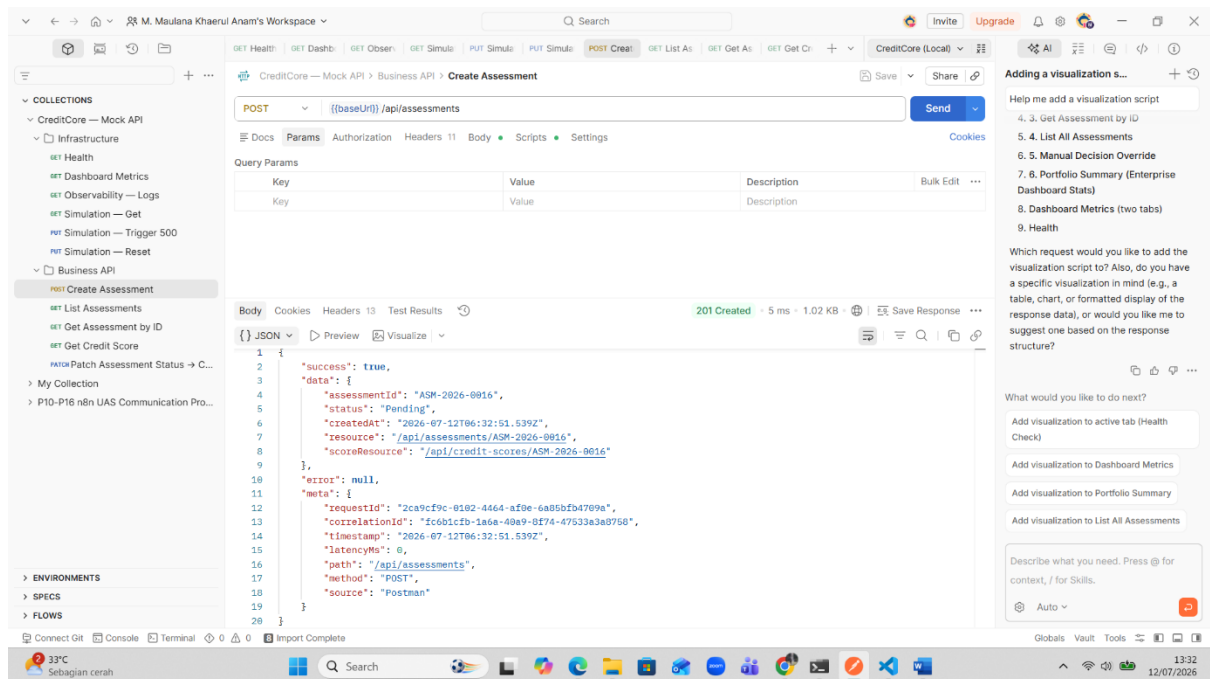
```
{
  "accountId": "ACC-2026-0042",
  "borrowerName": "Anam",
  "requestedAmount": 10000000,
  "monthlyIncome": 15000000,
  "employmentLengthMonths": 24,
  "purposeOfLoan": "Business",
  "numberOfDependents": 2,
  "slikStatus": "Kolektibilitas"
}
```

Response

```
{
  "assessmentId": "ASM-2026-0015",
  "creditScore": 764,
  "riskGrade": "LOW",
  "decision": "APPROVED"
}
```

Success Scenario

Success 1 — POST /api/assessments

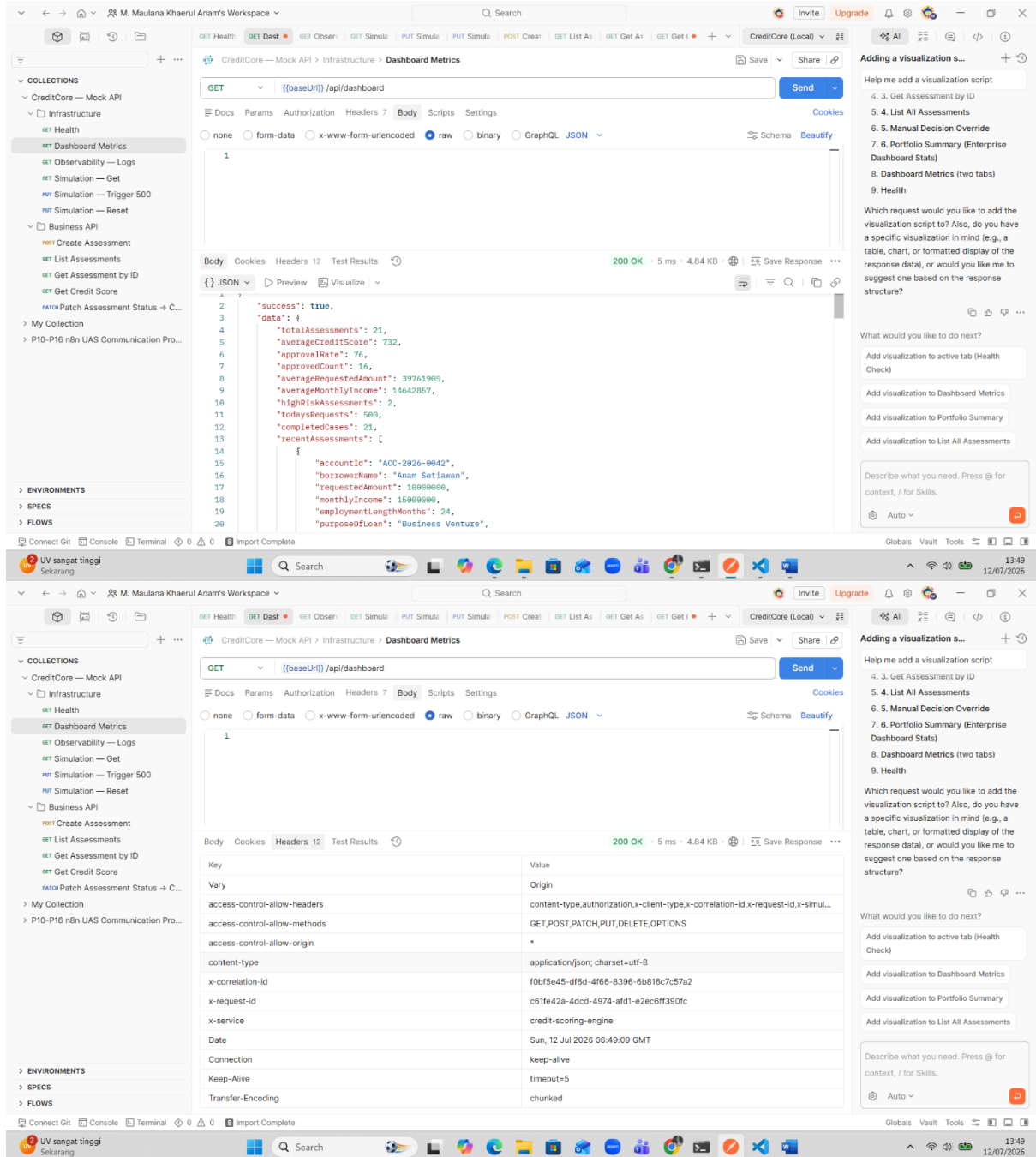


Keterangan berdasarkan SS gambar

- Untuk status code = success true, 201 created artinya membuat assessments baru berhasil dengan method post
- Error contract Null = kosong
- Source = postman (bisa dari demo app) tergantung jalan dimana

- Content type = application json
- Time out = 5ms

Success 2 — GET /api/dashboard



Keterangan berdasarkan SS gambar

- Untuk status code = success true,200 artinya berhasil mengambil data dashboard dengan method get
- Error contract Null= kosong

- Source = postman (bisa dari demo app)tergantung jalan dimana
- Content type = application json
- Sevice = credit-scoring-engine
- Time out = 5ms

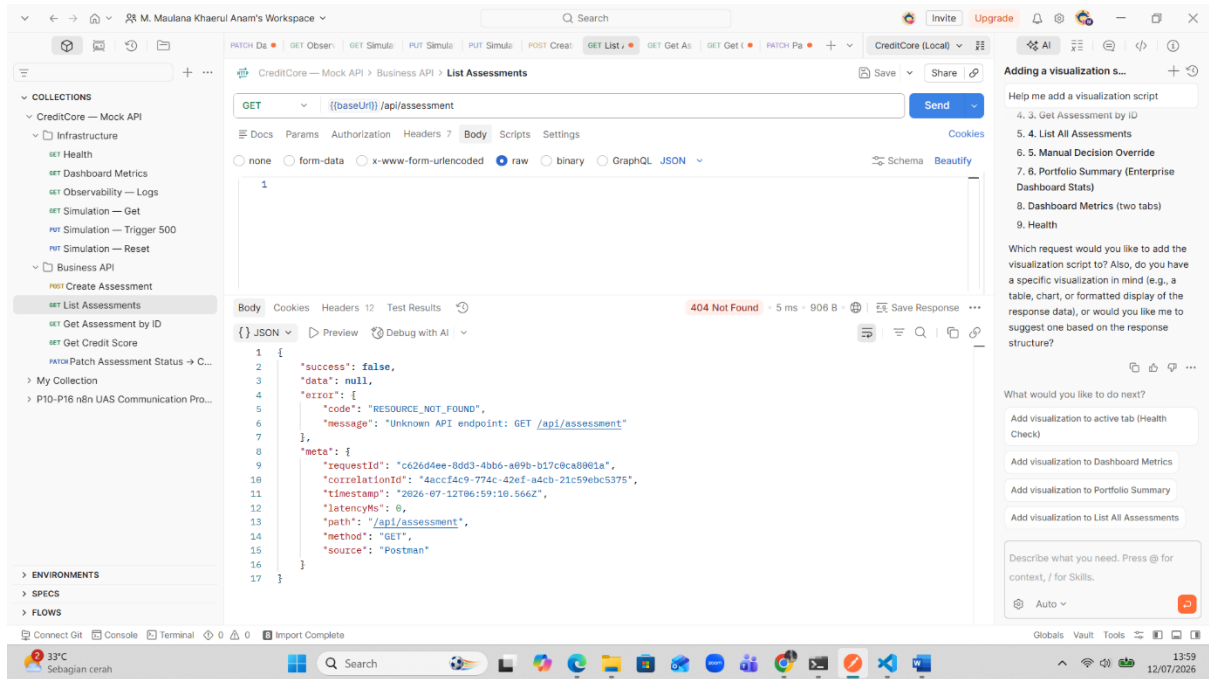
Failure Scenario

Failure 1 — GET assessment dengan resource tidak sesuai

The screenshot shows a Postman interface with a GET request to `{{baseUrl}}/api/assessment` under the 'CreditCore' workspace. The request is configured with 'raw' body type and 'JSON' schema. The response status is '404 Not Found' with a 5 ms duration and 906 B size. The 'Headers' tab is selected, showing various headers including 'access-control-allow-headers', 'access-control-allow-methods', 'access-control-allow-origin', 'content-type', 'x-correlation-id', 'x-request-id', 'x-service', 'Date', 'Connection', 'Keep-Alive', and 'Transfer-Encoding'.

Key	Value
Vary	Origin
access-control-allow-headers	content-type,authorization,x-client-type,x-correlation-id,x-request-id,x-simul...
access-control-allow-methods	GET,POST,PATCH,PUT,DELETE,OPTIONS
access-control-allow-origin	GET,POST,PATCH,PUT,DELETE,OPTION
content-type	application/json; charset=utf-8
x-correlation-id	4accf4c9-774c-42ef-a4cb-21c59ebc5375
x-request-id	c626d4ee-8dd3-4bb6-a09b-b17c0ca8001a
x-service	credit-scoring-engine
Date	Sun, 12 Jul 2026 06:59:10 GMT
Connection	keep-alive
Keep-Alive	timeout=5
Transfer-Encoding	chunked

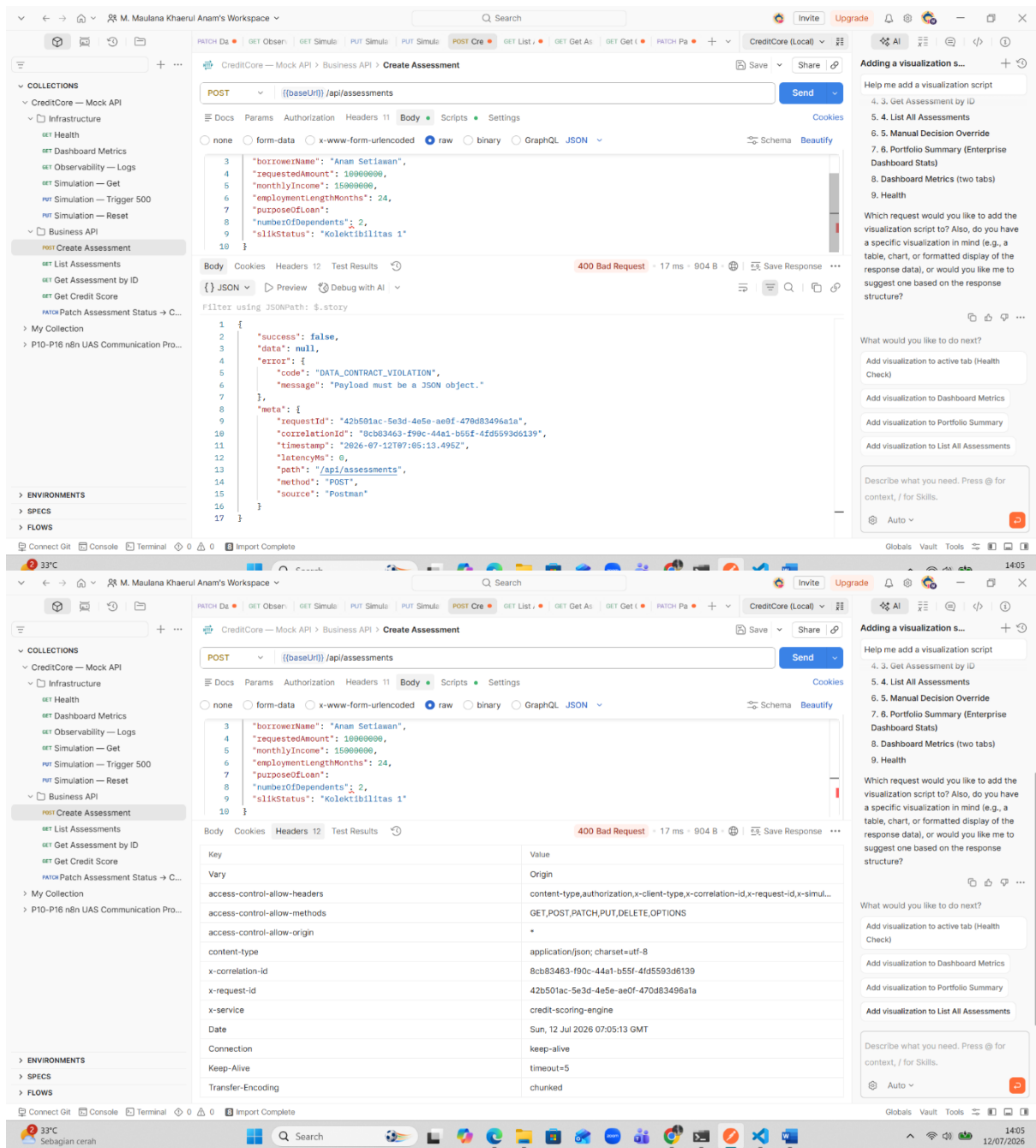
On the right side, there is a sidebar with 'Adding a visualization s...' and a list of suggestions for visualization scripts, including '4. 3. Get Assessment by ID', '5. 4. List All Assessments', '6. 5. Manual Decision Override', '7. 6. Portfolio Summary (Enterprise Dashboard Stats)', '8. Dashboard Metrics (two tabs)', and '9. Health'.



Keterangan berdasarkan SS gambar

- Untuk status code = false,404 artinya resource/endpoint tidak ditemukan
- Error contract = tidak ada endpoint,harusnya api/assessments ada s dibelakangnya
- Source = postman (bisa dari demo app)tergantung jalan dimana
- Content type = application json
- Sevice = credit-scoring-engine
- Time out = 5ms

Failure 2 — POST assessment dengan data tidak lengkap



Keterangan berdasarkan SS gambar

- Untuk status code = 400 artinya bad request ada content type, payload yg tidak lengkap
- Error contract = karena field request json tidak lengkap, sehingga respon nya bad request
- Source = postman (bisa dari demo app) tergantung jalan dimana
- Content type = application json
- Service = credit-scoring-engine

- Time out = 17 ms

Observability Evidence

Sistem menyediakan observability melalui endpoint:

GET /api/observability/logs

The screenshot displays a REST client interface with the following components:

- Endpoint:** GET /api/observability/logs
- Response Status:** 200 OK, 36 ms, 2.53 MB
- Response Body (JSON):**

```
{  "id": "ebee04f1-71d5-49cc-8242-125eed61b572",  "requestId": "0ef70e0e-b6f4-41bf-84f9-77d8d92dfc7",  "correlationId": "ccf5d812-cc46-48a9-91c9-331bbcb3b66",  "timestamp": "2026-07-12T07:09:44.636Z",  "method": "GET",  "path": "/api/observability/logs",  "status": 200,  "latencyMs": 0,  "source": "Lovable UI",  "clientType": "Lovable UI",  "userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0 Safari/537.36 Edg/150.0.0",  "responseType": "json",  "requestHeaders": {    "accept": "application/json",    "accept-encoding": "gzip, deflate, br, zstd",    "accept-language": "en-US,en;q=0.9",    "connection": "keep-alive",    "cookie": "rl_page_init_referrer=RudderEncryptK3AU2FsdGVkX19hSnpV1V6945DyS7j4hhzBT10Wck2B1cK3D; v1_nada_intc_waferring_domain=0uddaefccruntK3AU2FsdGVkX19hSnpV1V6945DyS7j4hhzBT10Wck2B1cK3D;"  },  "cookies": {  }}
```
- Headers Tab:** Shows various headers including access-control-allow-headers, access-control-allow-methods, access-control-allow-origin, content-type, x-correlation-id, x-request-id, x-service, Date, Connection, Keep-Alive, and Transfer-Encoding.

Informasi yang dicatat meliputi:

Request ID = " 0af70e8e-b6f4-41bf-84f8-77d0d920dfc7"

Correlation ID = "ccf5d812-cc46-40a9-91c9-331bbcb3b66"

Timestamp = "2026-07-12T07:09:44.636Z",

- Status Code = 200 ok,success true
- Response Time = 36 ms
- HTTP Method = GET
- Endpoint = api/observability/logs
- Source Client = loveble UI
-

Wireshark

The image shows a Wireshark packet capture of a GET request to /api/observability/logs. The packet list on the left shows a sequence of TCP and HTTP packets. The selected packet (No. 13) is a GET request from 10.0.0.1 to 10.0.0.1 on port 8080. The packet details pane shows the following information:

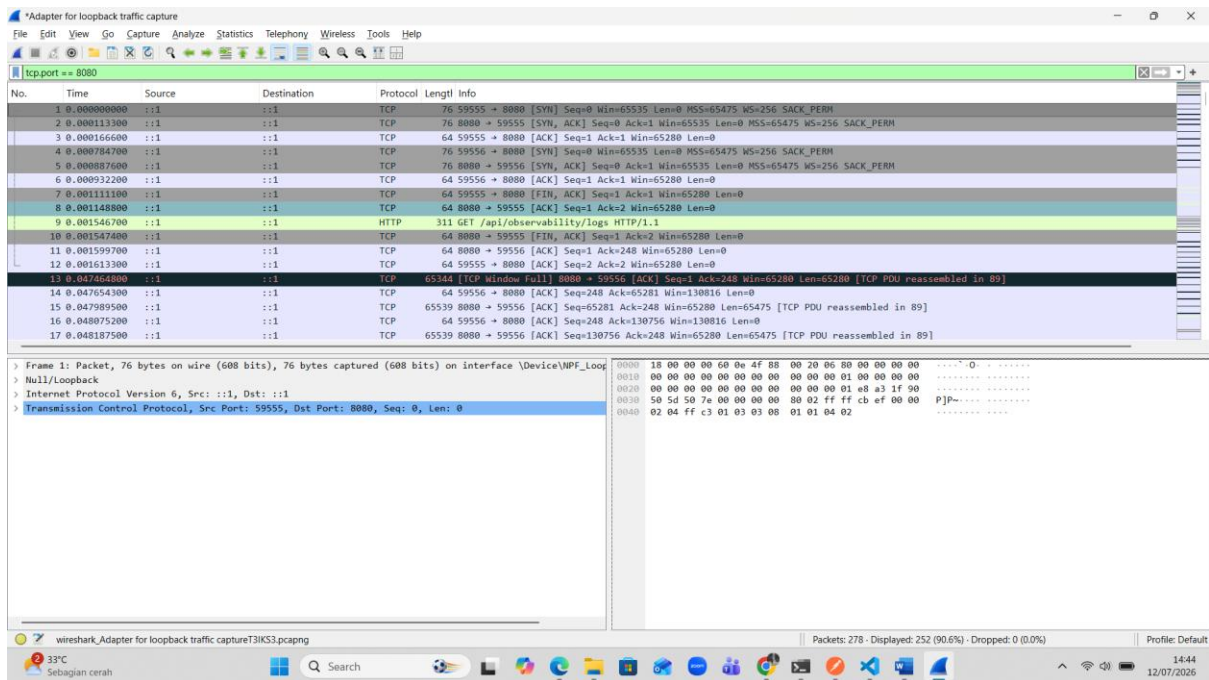
- Frame 9: Packet, 311 bytes on wire (2488 bits), 311 bytes captured (2488 bits) on interface (Device\NPF{...})
- Null/Loopback
- Internet Protocol Version 6, Src: ::1, Dst: ::1
- 0110 = Version: 6
- 0000 0000 = Traffic Class: 0x00 (DSCP: CS0, ECN: Not-ECT)
- 0000 00.. = Differentiated Services Codepoint: Default (0)
-00 = Explicit Congestion Notification: Not ECN-Capable Transport
- Payload Length: 267
- Next Header: TCP (6)
- Hop Limit: 128
- Source Address: ::1
- Destination Address: ::1
- [Stream index: 0]
- Transmission Control Protocol, Src Port: 59556, Dst Port: 8080, Seq: 1, Ack: 1, Len: 247
- Hypertext Transfer Protocol

The packet bytes pane shows the raw data of the packet, including the GET request line and the body.

The image shows a Wireshark packet capture of a GET request to /api/observability/logs. The packet list on the left shows a sequence of TCP and HTTP packets. The selected packet (No. 13) is a GET request from 10.0.0.1 to 10.0.0.1 on port 8080. The packet details pane shows the following information:

- [Time since first frame in this TCP stream: 762.000 microseconds]
- [Time since previous frame in this TCP stream: 614.500 microseconds]
- [SQ/ACK analysis]
- [Client Contiguous Streams: 1]
- [Server Contiguous Streams: 1]
- TCP payload (247 bytes)
- Hypertext Transfer Protocol
- GET /api/observability/logs HTTP/1.1\r\n
- x-client-type: Postman\r\n
- User-Agent: PostmanRuntime/7.54.0\r\n
- Accept: */*\r\n
- Postman-Token: 65318f6e-93ed-4feb-acad-a55d73de52e\r\n
- Host: localhost:8080\r\n
- Accept-Encoding: gzip, deflate, br\r\n
- Connection: keep-alive\r\n
- \r\n
- [Response in frame: 89]
- [Full request URI: http://localhost:8080/api/observability/logs]

The packet bytes pane shows the raw data of the packet, including the GET request line and the body.



Port **8080** digunakan oleh server CreditCore Mock API sehingga seluruh komunikasi HTTP antara Postman dan FastAPI dapat ditangkap.

No	Packet	Keterangan
1	SYN	Client (Postman) meminta koneksi ke server
2	SYN ACK	Server menerima permintaan koneksi
3	ACK	Client mengkonfirmasi koneksi berhasil

Dia ngirim GET /api/observability/logs HTTP/1.1

Header yg ditangkap :

Host: localhost:8080

User-Agent: PostmanRuntime

Accept: */*

Connection: keep-alive

Source port & Destination :

Source : ::1

Destination : ::1

Karena server dijalankan pada komputer yang sama (**localhost**), maka seluruh paket dikirim dan diterima melalui loopback interface sehingga source dan destination sama-sama menggunakan alamat **::1**.

Payloadnya :

GET /api/observability/logs HTTP/1.1

Host: localhost:8080

User-Agent: PostmanRuntime

Accept: */*

Testing Result dan Reliability Analysis

Hasil Pengujian

Pengujian	Hasil
Create Assessment	Berhasil
Get Assessment	Berhasil
Dashboard	Berhasil
Credit Score	Berhasil
Observability	Berhasil
Health Check	Berhasil
Error Testing	Berhasil

Reliability

Sistem mampu memberikan response sesuai kontrak API. Status code yang berhasil diuji meliputi:

- 200 OK
- 201 Created
- 400 Bad Request
- 404 Not Found

Setiap request menghasilkan audit log yang berisi Request ID, Correlation ID, response time, serta status code sehingga memudahkan proses monitoring dan debugging.

Kesimpulan

Mini project berhasil mengimplementasikan komunikasi menggunakan REST API berbasis HTTP dan JSON. Seluruh endpoint dapat diakses melalui browser maupun Postman dengan hasil yang konsisten. Sistem juga menyediakan observability melalui audit log sehingga aktivitas komunikasi dapat dipantau secara real-time.

Limitation

- Repository masih menggunakan In-Memory Storage, sehingga data akan hilang saat server dihentikan.
- Belum menggunakan database seperti PostgreSQL atau MySQL.
- Belum menerapkan autentikasi dan otorisasi.
- Belum menggunakan HTTPS pada lingkungan pengembangan.

Improvement

- Mengintegrasikan database PostgreSQL sebagai penyimpanan permanen.
- Menambahkan autentikasi menggunakan JWT.
- Menerapkan Docker untuk deployment.
- Mengembangkan workflow otomatis menggunakan n8n.
- Menambahkan rate limiting dan monitoring yang lebih komprehensif.

Reflection

Melalui mini project ini saya memahami implementasi komunikasi protokol menggunakan REST API, proses pengujian endpoint dengan Postman, serta pentingnya observability dalam memantau aktivitas komunikasi antar sistem. Selain itu, saya juga mempelajari bagaimana merancang arsitektur backend sederhana yang dapat dikembangkan menjadi sistem yang lebih kompleks.

GitHub Repository

<https://github.com/khaerulanamm/creditscorehub>

Video Demo (Unlisted)

<https://youtube.com/xxxxxxx>